GRADESENSE TECHNICAL ASSIGNMENT

Build a reliable grading and annotation tool

A practical assignment for the AI/ML Product Engineering Intern role

GOAL	**Build a small tool that reads a student answer, compares it with a model answer and marking rubric, gives an explainable score, and shows mistakes directly on the answer paper.**

The problem

AI grading is not only about generating a score. The result should be clear, consistent, and easy for a teacher to check. A teacher should be able to see what the student wrote, where the problem is, how many marks were lost, and what would improve the answer.

What we will provide

- One short question paper.
- One model-answer and marking-rubric file.
- A blank answer-sheet template.
- A small set of extra test cases for checking your system.

What you need to create

- Write a realistic 1-2 page student answer using the question paper.
- Intentionally include a mix of correct points, missing points, wrong reasoning, spelling or grammar errors, and layout or alignment problems.
- Make the answer difficult but believable. Do not make it random or unreadable.
- Submit a short error key showing each intended mistake and its correct version.

What you need to build

Part	Minimum expectation
Upload	Upload and read the question paper, answer paper, and model answer/rubric.
Grading	Give marks for each rubric point and calculate the final score.
Explanation	Show evidence from the student answer, missing or wrong points, and specific feedback.
Annotation	Underline or box the problem at the correct place and show the correction nearby.
Editable output	Annotations must remain editable: a user should be able to move, change, or delete an annotation without grading the paper again. Export an annotated copy as a PDF.
Reliability	Handle blank answers, unclear answers, model/API failures, malformed output, and marks above the limit.
History	Save the grading result so it can be viewed again.

Expected grading result

For every answer, the result should include:

- Total marks and maximum marks.
- Marks for each rubric point.
- Evidence from the student's answer.

- Correct, missing, and incorrect points.
- Specific correction or feedback.
- A confidence value and a human-review flag when the result is uncertain.

Important rules

- Marks must never be higher than the marks available.
- The total must equal the sum of the rubric-point marks.
- Feedback must be supported by evidence from the student answer.
- The original answer paper must not be destroyed or changed; create an annotated copy.
- If the system is uncertain, it should say so instead of pretending to be correct.

Technical expectations

Use any reasonable technology. React and TypeScript with a Node.js backend are preferred, but we care more about your decisions and the quality of the result than about using every tool in our stack.

- Use an LLM API or a mock model. If you use an API, do not commit API keys.
- Use any local database or simple persistence method.
- You may use AI coding tools such as Claude or Codex. You must understand and explain the code you submit.
- Do not spend time building login, complex design systems, fine-tuning, or production cloud infrastructure.

Tests

Include tests for at least these cases:

- A fully correct answer.
- A partially correct answer.
- An incorrect answer.
- A blank answer.
- An answer with OCR-like spelling errors.
- Malformed or incomplete model output.
- A model/API failure.
- An answer where the score would exceed the maximum unless it is corrected.

What to submit

- Source code in a Git repository or ZIP file.
- A README with setup and run instructions.
- A short architecture explanation.
- Your tests and their output.
- One example of an annotated answer paper.
- A short 2 min video explaining and showcasing what you have built.

Time limit

You have 36 hours to submit. A complete, reliable solution is more valuable than a large solution with unfinished features.